

From Code to Computation

Code Crafting no. 1

**From Code to Computation:
A Modern Guide to
Programming and Theory**

**Set Lonnert
with AI Coöperation**

All trademarks are the property of their respective owners.

All images are in the public domain unless otherwise stated.

Cover illustration in: Journal for Manufactures and Household Management, Scheutz, Stockholm, 1825, third issue, between pp. 96–97. Uppsala University Library. <https://urn.kb.se/resolve?urn=urn:nbn:se:alvin:portal:record-95006>

An illustration of the Jacquard mechanism. A device that uses punched cards to control the lifting of individual warp threads on a loom, making it possible to automate the weaving of intricate patterns with precision.

To the extent possible under law, Set Lonnert has waived all copyright and related or neighbouring rights to this work, using the Creative Commons CC0 1.0 Universal Public Domain Dedication.

This work is marked with CC0 1.0 Universal.

To view a copy of this license, visit:

<https://creativecommons.org/publicdomain/zero/1.0/>

Author: Set Lonnert 2026

AI Support: Multiple large language models

Publisher: BoD, Books on Demand, Östermalmstorg 1, 114 42 Stockholm, Sweden, bod@bod.se

Printer: Libri Plureos GmbH, Friedensallee 273, 22763 Hamburg, Germany

ISBN: 978-91-8114-246-4

Contents

Introduction	i
1 Foundations	1
1.1 Foundations of Programming	1
1.2 Simple Data Types	2
1.2.1 Integers in Binary	2
1.2.2 Floating-point Numbers	3
1.2.3 Characters and ASCII	4
1.2.4 Strings	4
1.2.5 Representations and Types	5
1.2.6 Type Systems and Type Checking	5
1.2.7 Summary	6
1.3 Variables	7
1.3.1 Assignment	7
1.3.2 Mutable and Immutable Variables	8
1.3.3 Summary	10
1.4 Control Structures	11
1.4.1 Conventional Control Structures	11
1.4.2 Control Structures in Computers	12
1.4.3 Summary	15
1.5 Functions	16
1.5.1 Calling Functions	16
1.5.2 Summary	18
1.6 Practice	18
2 Understanding VMs	21
2.1 Prerequisites	21
2.2 Simple VMs	21
2.2.1 The Stack	22
2.2.2 Interpreter technique	23

2.2.3	VM1 Implementation	24
2.2.4	REGVM Implementation	25
2.2.5	Portability	29
2.2.6	Summary	30
2.3	Stack-based VM	31
2.3.1	Comparisons	33
2.3.2	Iterations	34
2.3.3	Error Handling	36
2.3.4	Summary	38
2.4	Memory and Functions	39
2.4.1	Frame Pointer	42
2.4.2	Local Storage	44
2.4.3	Memory Management	45
2.4.4	Frame stack	46
2.4.5	Summary	51
2.5	Practice	53
3	Development Environment	57
3.1	Prerequisites	57
3.2	Basic tools	57
3.3	Debugging	58
3.3.1	Process	60
3.3.2	Root Cause Analysis	62
3.3.3	Tools	65
3.3.4	Distributed and Concurrent Systems	72
3.3.5	Summary	78
3.4	Optimisation	78
3.4.1	Memory	83
3.4.2	Performance	85
3.4.3	Confusion Matrix	87
3.4.4	Summary	93
3.5	Tests and Testing	93
3.5.1	Automated Testing and Continuous Integration	98
3.5.2	A Custom Test Virtual Machine	102
3.5.3	Summary	103
3.6	Advanced Debugging Practices	104
3.6.1	Error Handling and Resilience	104
3.6.2	Production and Live System Debugging	110
3.6.3	Summary	123
3.7	The Impact of LLMs	123
3.7.1	Debugging	124

3.7.2	Optimisation	124
3.7.3	Testing	125
3.7.4	Broader Perspectives and Challenges	126
3.7.5	Summary	126
3.8	Practice	127
4	Building and Experimenting	131
4.1	Prerequisites	131
4.2	The Computer as Hardware	131
4.2.1	Hardware	133
4.2.2	The Pico	135
4.2.3	Summary	138
4.2.4	Practice	139
4.3	Basic Input/Output	141
4.3.1	The Pico Pins	142
4.3.2	Light Switching Circuit	143
4.3.3	Programmable I/O	145
4.3.4	State Machines	147
4.3.5	Circuit for Traffic Lights	150
4.3.6	Pedestrian Crossing	152
4.3.7	Temperature Measurements	154
4.3.8	Temperature Indicator	155
4.3.9	Summary	160
4.3.10	Practice	162
4.4	Secondary Memory	163
4.4.1	Save and Load Temperatures	166
4.4.2	A Simple Database on SD Card	167
4.4.3	Summary	170
4.4.4	Practice	170
4.5	External Communication	171
4.5.1	Universal Asynchronous Receiver-Transmitter	171
4.5.2	Inter-Pico Communication	176
4.5.3	Client-Server Architecture	180
4.5.4	Wireless Communication	186
4.5.5	WiFi Communication Between Pico W Boards	188
4.5.6	Advanced WiFi Features	193
4.5.7	Features	196
4.5.8	Network Protocol Stack Integration	197
4.5.9	Socket Programming	198
4.5.10	Summary	203
4.5.11	Practice	204

4.6	Cryptography and Secure Communication	205
4.6.1	Core Principles	205
4.6.2	Symmetric vs Asymmetric Encryption	205
4.6.3	Symmetric Encryption	206
4.6.4	Asymmetric Encryption	207
4.6.5	Data Integrity and Hashing	214
4.6.6	Hybrid Cryptographic Systems	215
4.6.7	Digital Signatures	218
4.6.8	Implementation Considerations	218
4.6.9	Summary	220
4.6.10	Practice	220
4.7	Graphics	222
4.7.1	Framebuffers and DMA	222
4.7.2	Raster and Vector Approaches	225
4.7.3	Display Interfaces	229
4.7.4	Pimoroni Pico Display Pack 2.0	230
4.7.5	Colour Models and Bit Depth	233
4.7.6	Rendering Strategies	234
4.7.7	Simple Double Buffering Example	235
4.7.8	Performance Considerations	236
4.7.9	Text Rendering	236
4.7.10	Special Techniques and Hardware	238
4.7.11	Summary	241
4.7.12	Practice	241
4.8	Handling errors	242
4.8.1	Error Types	242
4.8.2	Approaches	243
4.8.3	Summary	246
4.8.4	Practice	247
4.9	Timing and timers	248
4.9.1	Timing and Timers in the Pico	249
4.9.2	Interrupt handling	251
4.9.3	Summary	252
4.9.4	Practice	252
4.10	Concurrency and multithreading	253
4.10.1	Summary	258
4.10.2	Practice	258
4.11	Power management	259
4.11.1	Summary	264
4.11.2	Practice	265
4.12	Practice	266

5	The Compiler Pipeline	271
5.1	Prerequisites	271
5.2	Compilers, Interpreters, Assemblers, and VMs	271
5.3	A Short History	274
5.4	The Pipeline in Principle	275
5.5	Syntax	276
5.5.1	Tokenisation or Lexical Analysis	277
5.5.2	Parsing	277
5.5.3	Production Rules and Grammars	280
5.5.4	Building Abstract Syntax Trees	281
5.5.5	Summary	281
5.6	Semantic Analysis	282
5.6.1	How Semantic Analysis Works	282
5.6.2	Symbol Table	283
5.6.3	Type System	284
5.6.4	Using Abstract Syntax Trees	285
5.6.5	Control Flow Graph	289
5.6.6	Example: Control Flow Graph	293
5.6.7	Data Flow and Dependency Analysis	297
5.6.8	Attribute Grammars	298
5.6.9	Consistency and Contract Checking	299
5.6.10	Summary	300
5.7	Intermediate Representations	300
5.7.1	Static Single Assignment	302
5.7.2	Three Address Code	305
5.7.3	Directed Acyclic Graphs	306
5.7.4	Summary	311
5.8	Code Generation Techniques	311
5.8.1	Example: Tail-Recursion	320
5.8.2	Summary	324
5.9	Loader and Linker	325
5.9.1	Linkers	325
5.9.2	Loaders	326
5.9.3	Real Examples	326
5.9.4	Simple Simulation	327
5.9.5	Summary	331
5.10	Optimisation in Compilers	332
5.10.1	Types of Compiler Optimisations	333
5.10.2	The Future of Compiler Optimisation	337
5.10.3	Summary	337
5.11	Error Handling	338

5.11.1	Warnings	338
5.11.2	Errors	339
5.11.3	Future of Error Handling	342
5.11.4	Summary	343
5.12	Practice	343
6	Philosophy and Methodology	353
6.1	Philosophy and Style	353
6.1.1	Programming as Craft	354
6.1.2	Building Process	355
6.1.3	Blending Philosophies	358
6.1.4	Methodologies	360
6.1.5	References	363
6.1.6	A Personal Note	365
6.1.7	Risks	367
6.1.8	Summary	368
6.2	Problems and Methods	369
6.2.1	Example: Low Customer Satisfaction	369
6.2.2	Example: Thermostat	369
6.2.3	Generalisations	371
6.2.4	Methodology in the Craft Philosophy	374
6.2.5	Summary	377
6.3	Reflections on The History and Future	378
6.3.1	AI Example: LLMs	380
6.3.2	Summary	383
6.4	Method Examples	383
6.4.1	Mockups and Prototypes	384
6.4.2	Code Review	388
6.4.3	Null Hypothesis	394
6.4.4	Null Hypothesis in Programming	396
6.4.5	Summary	399
6.5	Practice	400
7	The Mechanics	403
7.1	Data: Foundations, Structures, and Algorithms	403
7.1.1	Elementary Data Types	404
7.1.2	Data Structures	404
7.1.3	Abstract Data Types	405
7.1.4	Algorithms	406
7.1.5	Algorithmic Paradigms	407
7.1.6	Summary	415

7.2	Types and Type Systems	416
7.2.1	Data Types	416
7.2.2	Type Systems	417
7.2.3	Type Errors	417
7.2.4	Static vs. Dynamic Typing	418
7.2.5	Type Inference	418
7.2.6	Type Consistency and Soundness	418
7.2.7	Strong vs. Weak Typing	419
7.2.8	Beyond Data Safety	419
7.2.9	Summary	419
7.3	Programming Paradigms and Language Design	420
7.3.1	Imperative Languages	421
7.3.2	Functional Languages	423
7.3.3	Object-Oriented Languages	426
7.3.4	Concatenative Languages	433
7.3.5	Homoiconicity and Metaprogramming	435
7.3.6	Domain-Specific Languages	436
7.3.7	Example: Rust	442
7.3.8	Summary	444
7.4	Low-Level Constructs	445
7.4.1	Callback	447
7.4.2	State Machine	448
7.4.3	Checkpoint	450
7.4.4	Low-Level Programming Concepts	451
7.4.5	Summary	453
7.5	Architecture in Programming	453
7.5.1	History	454
7.5.2	Styles and Patterns	454
7.5.3	Change	456
7.5.4	Code Architecture vs. System Architecture	457
7.5.5	Practical Considerations in Modern Architectures	457
7.5.6	Summary	458
7.6	Design Patterns	458
7.6.1	Factory	460
7.6.2	Observer	462
7.6.3	Command	465
7.6.4	Strategy	467
7.6.5	Adapter	470
7.6.6	Composite	473
7.6.7	Decorator	475
7.6.8	Summary	477

7.7	Networking and Modern Server Architectures	478
7.7.1	Server Systems	479
7.7.2	A Protocol Suite: TCP/IP	481
7.7.3	Web Servers	489
7.7.4	Application-Level Protocols and Styles	490
7.7.5	System-Level Architectural Patterns	492
7.7.6	Concurrency and Advanced Server Functions	496
7.7.7	Summary	499
7.8	Practice	500
8	Perspectives and Frontiers	503
8.1	Systemic Concepts in Computing	503
8.1.1	Abstraction	504
8.1.2	Latency	504
8.1.3	Resilience	505
8.1.4	Summary	506
8.2	Computer Security	508
8.2.1	The Modern Threat Landscape	509
8.2.2	Fundamental Principles	510
8.2.3	The Security Ecosystem	511
8.2.4	Human Factors	512
8.2.5	The Role of Cryptography	513
8.2.6	Principles of Cryptography	513
8.2.7	Cryptographic Techniques	514
8.2.8	Challenges in Cryptographic Security	515
8.2.9	Summary	516
8.3	Exploring the Limits of Computation	516
8.3.1	Summary	522
8.4	The Turing Machine	523
8.4.1	The Halting Problem	524
8.4.2	Proof of Undecidability	525
8.4.3	Visualising the Proof	526
8.4.4	Implications	528
8.4.5	Summary	528
8.5	Information Theory	528
8.5.1	Quantifying Information	529
8.5.2	Entropy	530
8.5.3	Joint, Conditional, and Mutual Information	530
8.5.4	The Limit of Communication	531
8.5.5	Data Compression and Source Coding	533
8.5.6	Divergence and Cross-Entropy	534

8.5.7	Summary	535
8.6	Artificial Intelligence and Machine Learning	535
8.7	Machine Learning	539
8.7.1	Three Samples	547
8.7.2	Summary	559
8.8	Deep Learning	559
8.8.1	Foundations	561
8.8.2	Training via Backpropagation	563
8.8.3	Convolutional Neural Networks (CNNs)	564
8.8.4	Recurrent Neural Networks (RNNs) and LSTMs	566
8.8.5	Transformers	567
8.8.6	Summary	568
8.9	Capabilities, Limitations, and Impacts of AI/ML	569
8.9.1	Limitations and Risks	571
8.9.2	Strengths and Achievements	578
8.9.3	Summary	579
8.10	AI: The Catalyst for a New Era in Programming	580
8.10.1	The Quality-Time Tradeoff	581
8.10.2	Rethinking Verification	582
8.10.3	Formal Approaches	582
8.10.4	Empirical Approaches	582
8.10.5	Summary	583
8.11	Hoare Logic	584
8.11.1	Inference Rules	585
8.11.2	Practical Examples	587
8.11.3	Advanced Topics	591
8.11.4	Proof Techniques: Loop Invariants & Failed Proofs	592
8.11.5	Tools and Applications	592
8.11.6	Limitations and Extensions	592
8.11.7	Summary	593
8.12	Property-Based Testing	593
8.12.1	The Idea	594
8.12.2	Properties vs. Examples: A Paradigm Shift	596
8.12.3	Foundations	597
8.12.4	Implementation Architecture	602
8.12.5	Advanced Concepts and Techniques	607
8.12.6	Practical Applications	616
8.12.7	Performance Considerations	627
8.12.8	Debugging and Diagnosis	629
8.12.9	Best Practices and Guidelines	634
8.12.10	Model-Based and Metamorphic Testing	634

CONTENTS

8.12.11	Future Directions	638
8.12.12	Summary	638
8.13	Type Systems and Formal Reasoning	639
8.13.1	Lambda Calculus: A Simplified Overview	640
8.13.2	Dependent Types	642
8.13.3	Basic Concepts	642
8.13.4	Type Theory Basics	643
8.13.5	A Simple Type Theory with Dependent Types	647
8.13.6	Practical Applications and Examples	649
8.13.7	Example: Natural Deduction	650
8.13.8	Summary	653
8.14	Practice	654
Glossary		657
Index		732